FlaPy: Mining Flaky Python Tests at Scale

Martin Gruber
BMW Group, University of Passau
Munich, Germany
martin.gr.gruber@bmw.de

Gordon Fraser
University of Passau
Passau, Germany
gordon.fraser@uni-passau.de

arXiv:2305.04793v1 [cs.SE] 8 May 2023

***Abstract*—Flaky tests obstruct software development, and studying and proposing mitigations against them has therefore become an important focus of software engineering research. To conduct sound investigations on test flakiness, it is crucial to have large, diverse, and unbiased datasets of flaky tests. A common method to build such datasets is by rerunning the test suites of selected projects multiple times and checking for tests that produce different outcomes. While using this technique on a single project is mostly straightforward, applying it to a large and diverse set of projects raises several implementation challenges such as (1) isolating the test executions, (2) supporting multiple build mechanisms, (3) achieving feasible run times on large datasets, and (4) analyzing and presenting the test outcomes. To address these challenges we introduce FlaPy, a framework for researchers to mine flaky tests in a given or automatically sampled set of Python projects by rerunning their test suites. FlaPy isolates the test executions using containerization and fresh execution environments to simulate real-world CI conditions and to achieve accurate results. By supporting multiple dependency installation strategies, it promotes diversity among the studied projects. FlaPy supports parallelizing the test executions using SLURM, making it feasible to scan thousands of projects for test flakiness. Finally, FlaPy analyzes the test outcomes to determine which tests are flaky and depicts the results in a concise table. A demo video of FlaPy is available at https://youtu.be/ejy-be-FvDY**

***Index Terms*—Flaky Tests; Flakiness Detection**

I. Introduction

Flaky tests are tests that behave non-deterministically, i.e., they can both pass and fail on the same software version. Test flakiness is a serious issue since it slows down development by requiring manual failure analysis and delaying test feedback when rerunning tests, and because it causes developers to lose trust in testing [1]. To create an understanding of test flakiness, researchers have conducted empirical studies by mining and investigating flaky tests in open-source Java projects [2]–[5]. However, the community is lacking mining tools for other programming languages that include features important for research, such as parallelization of test executions.

To promote and enable research on flakiness in other languages, we developed *FlaPy*, a tool for mining flaky Python tests at scale by repeatedly executing the test suites of a given set of Python projects. Its core features are:

- Automatically sampling projects from PyPI [6]
- Isolating test executions for realistic and accurate results
- Supporting different dependency installation strategies to promote diversity within the set of sample projects
- Parallelizing test executions via SLURM [7]
- Analyzing and presenting results in a concise format

II. Using FlaPy

FlaPy is an easy to use tool that allows researchers to identify flaky tests within a given set of Python projects and their test suites. FlaPy itself is also written in Python and provides a command-line interface to execute tests and to parse the outcomes. Provided with a set of Python projects as input (Section II-B), it sets up an isolated executed environment and runs the projects' test suites a specified number of times (Sections II-C and II-D). FlaPy analyzes the resulting test outcomes and presents them in a concise table (Section II-E).

A. Installation

FlaPy uses Docker [8] to isolate the test executions and to provide a swift installation experience. To automate the creation of containers, it comes with supporting scripts which are located in the FlaPy repository and can be cloned via

```
git clone https://github.com/se2p/flapy
```

FlaPy's main entry point is the script `flapy.sh`. The FlaPy Docker image will be pulled automatically on first usage. Note that FlaPy requires Docker to be installed and executable without root privileges. Apart from that it has no requirements.

B. Input

After installing FlaPy, we can define our study subjects, i.e., the set of Python projects whose test suites we want to execute. As a running example throughout this paper, we use the following projects, in which we previously found flakiness:

- **avwx-engine**: a global aviation weather fetching and parsing engine.
- **jgutils**: a Python utility module.
- **flapy_example**: an example project created by the authors of this paper, which is located in the FlaPy repository under `minimal_flaky_python_tests/`

For each project, we have to provide a name and a Git-URL from which its sources can be cloned. Alternatively, we can also provide a path from which the sources will be copied. Optionally we can specify a Git revision or tag to be checked out before the test execution, a PyPI [6] version used for installing the project itself to the test environment to retrieve its dependencies instead of searching for dependencies in its source code, the functions we want to trace (see the documentation for more information), as well as a subset of tests we want to execute instead of the entire test suite. This information must be provided via a CSV file, which we call

Table I: Input-CSV for our running example.

PROJECT_NAME	PROJECT_URL	PROJECT_HASH	PYPI_TAG	FUNCS_TO_TRACE	TESTS_TO_RUN
avwx-engine	https://github.com/avwx-rest/avwx-engine	ed1d3e			
avwx-engine	https://github.com/avwx-rest/avwx-engine	ed1d3e			
avwx-engine	https://github.com/avwx-rest/avwx-engine	ed1d3e			
avwx-engine	https://github.com/avwx-rest/avwx-engine	ed1d3e			
jgutils	https://github.com/jerodg/jgutils	6c8ed6			
jgutils	https://github.com/jerodg/jgutils	6c8ed6			
jgutils	https://github.com/jerodg/jgutils	6c8ed6			
jgutils	https://github.com/jerodg/jgutils	6c8ed6			
flapy_example	./minimal_flaky_python_tests				test_flaky.py
flapy_example	./minimal_flaky_python_tests				test_flaky.py
flapy_example	./minimal_flaky_python_tests				test_flaky.py
flapy_example	./minimal_flaky_python_tests				test_flaky.py

the *input-csv*. A possible input-csv for our running example is depicted in Table I and can also be found in the FlaPy repository under the name `flapy_input_example.csv`. FlaPy can also generate input-csv files automatically by sampling projects from PyPI [6]. To do so, use `./flapy.sh sample`.

We refer to one row in the input-csv as an *iteration*. As Table I shows, in our example we do not use one, but four iterations per project. Splitting the test executions like this has several advantages, particularly when executing the tests on a SLURM cluster (Section II-D): Since each row in the input-csv results in one SLURM job, fine-grained chunking helps to avoid timeouts (many clusters have maximum runtimes per job, e.g., 24h) and allows for more parallelization and better scheduling, speeding up the process. Furthermore, since each SLURM job is potentially scheduled on a different machine, it helps to detect infrastructure flakiness [9], i.e., global issues that lead to bulked test failures.

C. Running Tests Locally

To run the test suites of our study subjects, we use the `run` command on `flapy.sh`, which offers the following interface:

```
./flapy.sh run [OPTIONS...] INPUT_CSV NUM_RUNS
```

It has two mandatory arguments, namely, the path to the input-csv file (`INPUT_CSV`), and the number of times the test suites should be executed (`NUM_RUNS`). Optionally, we can tell FlaPy to conduct additional test executions in random order (`--plus-random-runs`), allowing it to also detect order-dependent flaky tests, or to write the results to a specific directory (`--out-dir OUT_DIR`). Suppose in our running example we want the tests to be executed five times both in same and in random order and the results to be written to `example_results` (folder will be created by a helper script):

```
./flapy.sh run --out-dir example_results \
  --plus-random-runs flapy_input_example.csv 5
```

This causes FlaPy to sequentially iterate over the input-csv and start one container for each iteration (row). Algorithm 1 describes the setup and test execution taking place inside the container (entry point `ITERATION`). Isolating the test execution is especially important when executing unknown projects, whose code might have unwanted or even malicious side-effects (such as creating cronjobs or deleting files). Furthermore, test case isolation promotes reproducibility and comparability to real-world CI systems, which also often use Docker.

Algorithm 1 Test execution core procedure.

```
1:  function ITERATION(proj_name, proj_url, proj_hash,
        pypi_tag, tests_to_run, num_runs, plus_rand
    )
2:      CLONE_OR_COPY_SOURCES(proj_url)
3:      CHECKOUT_REVISION(proj_hash)
4:      MEASURE_LINES_OF_CODE_AND_DISK_USAGE()
5:      for i = 0; i < num_runs, i++ do
6:          RUN(proj_name, pypi_tag, tests_to_run, same)
7:      end for
8:      if plus_rand then
9:          for i = 0; i < num_runs, i++ do
10:             RUN(proj_name, pypi_tag, tests_to_run, random)
11:         end for
12:     end if
13:     ARCHIVE_RESULTS()
14: end function
15:
16: function RUN(proj_name, pypi_tag, tests_to_run, order)
17:     COPY_PROJECT_SOURCES()
18:     CREATE_VIRTUAL_ENVIRONMENT()
19:     if pypi_tag == NULL then
20:         SEARCH_AND_INSTALL_DEPENDENCIES()
21:     else
22:         INSTALL_PROJECT(proj_name, pypi_tag)
23:     end if
24:     PYTEST(tests_to_run, order)
25:     DELETE_COPY_AND_VIRTUALENV()
26: end function
```

One execution of `"./flapy.sh run"` creates one new folder within the `OUT_DIR` (`example_results/`), which we refer to as a *results-directory*. Results-directories have the following naming schema: "flapy-results_DATE_TIME", where DATE and TIME describe the point in time when the execution started. The results-directory created by our example execution is `flapy-results_20221123_161945`. Its structure is depicted in Fig. 1. Within a results-directory, FlaPy creates one sub-directory for each row in the input-csv, which we refer to as an *iteration-directory*. Each iteration-directory is named as follows: "ProjectName_DATE_TIME_RowNum", where RowNum is the line number of the iteration within the input-csv. This helps to differentiate iterations of the same project

```
example_results                      # OUT_DIR
`-- flapy-results_20221123_161945    # results-dir
    |-- !flapy.run
    |   |-- flapy_run.yaml
    |   `-- input.csv
    |
    |-- avwx-engine_20221123_161945_2       # iteration-dir
    |   |-- flapy-iteration-result.yaml
    |   |-- loc.csv
    |   `-- results.tar.xz
    |       | -- .../avwx-engine_output0.xml
    |       | -- .../avwx-engine_coverage0.xml
    |       | -- .../avwx-engine_output1.xml
    |       ...
    |
    |-- avwx-engine_20221123_161945_3
    |-- avwx-engine_20221123_161945_4
    |-- avwx-engine_20221123_161945_5
    |-- flapy_example_20221123_161945_10
    |-- flapy_example_20221123_161945_11
    |-- flapy_example_20221123_161945_12
    |-- flapy_example_20221123_161945_13
    |-- jgutils_20221123_161945_6
    |-- jgutils_20221123_161945_7
    |-- jgutils_20221123_161945_8
    `-- jgutils_20221123_161945_9
```

Figure 1: Output structure for running example.

within the same results-directory. One such iteration-directory in our example is `avwx-engine_20221123_161945_2`.

Each iteration-directory contains the following files:

- `flapy-iteration-result.yaml`: Input- and meta-information about this iteration.
- `loc.csv`: The lines of code of this project per programming language, measured via CLOC [10]. (The raw CLOC sqlite database can be found in `results.tar.xz`)
- `results.tar.xz`: An archive containing the test results (junit-xml and coverage-xml files)

Section II-E discusses how to use FlaPy's results-parser to generate an overview of the outcomes of all test executions.

D. Running Tests on SLURM

SLURM [7] is a popular open-source cluster management and job scheduling system, which can be used to execute multiple jobs in parallel across different machines. When executing FlaPy on a SLURM cluster, we highly recommend the usage of Podman instead of Docker. Podman [11] is a Docker alternative that provides the same interface, but allows rootless container executions and parallel non-blocking pulling of images, which happens when multiple FlaPy jobs run in parallel on the same machine. The use of Podman instead of Docker can be configured in `setup_docker_command.sh`, which is sourced before any Docker commands are invoked, or by setting the `FLAPY_DOCKER_COMMAND_SETUP_SCRIPT` environment variable. This can also be used to configure other cluster-specific setup steps, for example, (un)setting or changing global variables such as `HOME` or `TMP`.

To conduct the test executions on a SLURM cluster, we add the option `--run-on cluster` and specify a subset of nodes on which the job execution shall be scheduled via `--constraint`:

```
./flapy.sh run --out-dir example_results \
  --plus-random-runs \
  --run-on cluster --constraint CONSTRAINT \
  flapy_input_example.csv 5
```

Make sure that `OUT_DIR` (`example_results`) is a shared storage that is accessible from all nodes.

E. Analyzing Results

With the `parse` command, FlaPy offers a convenient way to analyze the test outcomes generated by the `run` command. To create an overview showing which tests have been executed and how often they yielded a certain verdict, we can use the following command for our running example:

```
./flapy.sh parse ResultsDirCollection \
  --path example_results \
  get_tests_overview _df \
  to_csv --index=False example_results_to.csv
```

Note that the directory specified after `--path` needs to be accessible from the current working directory since only the current working directory is mounted to the container that is started in the background. The generated *tests-overview* is saved to `./example_results_to.csv`. Table II depicts its content. Each row in the tests-overview corresponds to a test found and executed by pytest. A test is uniquely identified by its name (Test_name), the name of its class (Test_classname), as well as possibly existing parametrizations (Test_parametrization). The eighth column (flaky?) contains FlaPy's verdict about if and how a test is flaky. It can take four different values:

- **not flaky**: no flaky behavior occurred, neither in same-order, nor in random-order runs.
- **non-order-dependent (NOD)**: flaky in same-order runs.
- **order-dependent (OD)**: flaky behavior in random-order runs, but not in same-order runs.
- **infrastructure**: flaky behavior, but only between iterations, never within. This suggests that a global issue occurred in some iterations that caused all runs of this test to fail.

The columns to the right of "flaky?" show the number of times each verdict was reached in same- and random-order (passed, failed, errored, or skipped). We consider a test to be flaky, iff it yielded at least one passing and one failing or erroring run. A test that alternates between failing and erroring runs is therefore not considered as flaky, since both outcomes would cause a build to fail and thus would not lead to the same developer experience as flakiness would cause.

Executing FlaPy on the three projects from our running example reveals nine flaky tests: six NOD and three OD. Note that tests marked as OD might actually be NOD, if the flakiness never unveiled itself in the same-order runs, as happened in the case of "`test_network_remote_connection_failure`". A recent study [12] showed that many NOD tests have very different failure rates for different orders.

Table II: Tests-overview summarizing the results of our running example (only flaky tests).

Omitted columns: Error_sameOrder, Skipped_sameOrder, Error_randomOrder, Skipped_randomOrder (all zero)

Project_Name	Project_URL Project_Hash Test_filename Test_classname Test_param.	Test_name	flaky?	Passed_ same Order	Failed_ same Order	Passed_ random Order	Failed_ random Order
avwx-engine	...	test_fetch	non-order-dependent	9	11	11	9
flapy_example	...	test_network_remote_connection_failure	order-dependent	20	0	18	2
flapy_example	...	test_numpy_random	non-order-dependent	10	10	13	7
flapy_example	...	test_random	non-order-dependent	15	5	8	12
flapy_example	...	test_time	non-order-dependent	12	8	9	11
flapy_example	...	test_unordered_collections	non-order-dependent	9	11	11	9
flapy_example	...	test_victim	order-dependent	20	0	14	6
jgutils	...	test_numeric_intermixed	non-order-dependent	11	9	8	12
jgutils	...	test_file_creation2	order-dependent	20	0	11	9

III. Evidence of Effectiveness

To demonstrate FlaPy's capabilities, we replicate our study on flaky tests in Python [9]: We randomly sample 30 000 projects from PyPI [6] using FlaPy's `sample` command:

```
./flapy.sh sample --sample-size 30000 > flapy_input.csv
```

The script randomly samples projects from PyPI, searches for GitHub URLs, and removes duplicates. Furthermore, it identifies PyPI tags for dependency installation wherever available an tries to match them with possibly existing Git tags to use the latter as the project hash. If no matching Git tag is found, the field remains empty and the latest commit is used. In our case, we retrieved a set of 26 451 cloneable projects, whose test suites we run 100 times in default- and 100 times in random order. Conducting the test executions on a SLURM cluster consisting of 58 machines, which allows us to run up to 700 iterations in parallel, takes about 3 days and generates 5 GB of compressed data. In comparison, the total (i.e., sequential) runtime is about 20 500 hours (854 days).

Among the 10 924 projects that contain at least one test, 396 also contain at least one flaky test. In total, we find 2 460 flaky tests in the roughly 1.5 million tests that were executed. 1 878 of them are OD, 543 are NOD, and 39 are flaky due to infrastructure. These numbers confirm prior results [9], except for less infrastructure flakiness. All data is accessible online [13]. We hope that this new dataset will continue serving researchers, as our first one already has [14]–[16].

IV. Extendability

FlaPy can be easily extended to implement new features, such as measuring further runtime metrics or manipulating the test execution to provoke flakiness. To do so, the `run` command offers the optional argument `--core-args CORE_ARGS` to forward arguments directly to the core test execution procedure, which is the script `flapy/run_tests.py`. To extend the core, one can therefore easily add command line options to this script and activate them via the `CORE_ARGS`. Note that since the arguments will be unpacked later, they should be passed as one string (e.g., `--core-args "--foo --bar"`). This could also be used to support projects written in languages other than Python. One option that is currently available is `--random-order-seed`, which sets the seed for the test case shuffling, in case of random order runs. After the core script has been modified, the FlaPy image needs to be rebuilt, which can be done via

```
docker build -t my_flapy -f Dockerfile .
```

This image can be used together with all existing scripts by changing the `FLAPY_DOCKER_IMAGE` variable in `setup_docker_command.sh` to `"localhost/my_flapy"`. When using this image in a SLURM execution, it needs to be pushed to a repository from which the cluster nodes can pull.

V. Related work

The iDFlakies framework [4] is similar to FlaPy, as it also conducts repeated containerized test executions based on a set of projects that can be specified via their Git-URL and -Hash. Unlike FlaPy, however, iDFlakies targets Java projects that are built using Maven or Gradle, and does not provide native support for parallelizing test executions.

Other rerun-based flakiness detection tools have been created by practitioners in the form of plugins for popular test execution frameworks [17]–[20]. However, these approaches are not targeting researchers aiming to conduct studies and evaluations, but developers, aiming to detect flaky tests within their own projects. Therefore, they have weaker requirements in regard to isolation (since the code is known) and parallelization. Aiming at the same goal, researchers proposed rerun-free flakiness detection approaches [3], [21]–[28]. FlaPy, on the other hand, targets researchers, who aim to conduct studies and evaluate their methods on large and diverse sets of sample projects.

VI. Conclusions

FlaPy is an easy to use tool to mine flaky tests by repeatedly executing the test suites of a given set of Python projects. It is designed to address the challenges that come with this task: By ensuring that the individual test runs are isolated, it simulates real-world CI conditions and ensures accurate results. To promote diversity among the sample projects, FlaPy supports different forms of dependency management. Furthermore, it provides means to parallelize the test executions across a SLURM cluster. Lastly, FlaPy offers an

easy-to-use interface to accumulate individual test outcomes into one concise table. FlaPy's source code is available at https://github.com/se2p/flapy under the GNU LGPL license. Contributions, feedback, and questions are most welcome.

References

[1] M. Gruber and G. Fraser, "A survey on how test flakiness affects developers and what support they need to address it," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 82–92.

[2] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 643–653.

[3] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *International Conference on Software Engineering (ICSE)*, 2018, pp. 433–444.

[4] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2019, pp. 312–322.

[5] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFixFlakies: A framework for automatically fixing order-dependent flaky tests," in *Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 545–555.

[6] "Python Package Index (PyPI)." [Online]. Available: https://pypi.org/

[7] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: simple linux utility for resource management," in *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, 2003, pp. 44–60.

[8] "Docker." [Online]. Available: https://www.docker.com/

[9] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An empirical study of flaky tests in Python," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2021, pp. 148–158.

[10] A. Danial, "cloc: v1.92," 2021. [Online]. Available: https://doi.org/10.5281/zenodo.5760077

[11] "Pod Manager tool (podman)." [Online]. Available: https://podman.io/

[12] W. Lam, S. Winter, A. Astorga, V. Stodden, and D. Marinov, "Understanding reproducibility and characteristics of flaky tests through test reruns in java projects," in *International Symposium on Software Reliability Engineering (ISSRE)*, 2020, pp. 403–413.

[13] M. Gruber and G. Fraser, "Flapy: Mining flaky python tests at scale [dataset]," 2023. [Online]. Available: https://doi.org/10.6084/m9.figshare.21610785

[14] W. L. Ruixin Wang, Yang Chen, "iPFlakies: A framework for detecting and fixing python order-dependent flaky tests," in *International Conference on Software Engineering: Companion Proceedings (ICSE Companion)*, 2022, pp. 120–124.

[15] A. Wei, P. Yi, Z. Li, T. Xie, D. Marinov, and W. Lam, "Preempting flaky tests via non-idempotent-outcome tests," in *International Conference on Software Engineering (ICSE)*, 2022, pp. 1730–1742.

[16] A. Ahmad, E. N. Held, O. Leifler, and K. Sandahl, "Identifying randomness related flaky tests through divergence and execution tracing," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 293–300.

[17] "Flaky - a plugin for nose or pytest that automatically reruns flaky tests." [Online]. Available: https://pypi.org/project/flaky/

[18] "Apache Maven: Rerun failing tests." [Online]. Available: https://maven.apache.org/surefire/maven-surefire-plugin/examples/rerun-failing-tests.ht

[19] E. Wendelin, "Introducing flaky test mitigation tools," 2020. [Online]. Available: https://blog.gradle.org/gradle-flaky-test-retry-plugin?utm_source=thenewstack&utm_medium=website&utm_campaign=platform

[20] "Bazel: Attribute flaky." [Online]. Available: https://bazel.build/reference/be/common-definitions

[21] T. M. King, D. Santiago, J. Phillips, and P. J. Clarke, "Towards a Bayesian network model for predicting flaky automated tests," in *International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2018, pp. 100–107.

[22] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *International Conference on Mining Software Repositories (MSR)*, 2020, pp. 492–502.

[23] E. Kowalczyk, K. Nair, Z. Gao, L. Silberstein, T. Long, and A. Memon, "Modeling and ranking flaky tests at Apple," in *International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, 2020, pp. 110–119.

[24] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "FlakeFlagger: Predicting flakiness without rerunning tests," in *International Conference on Software Engineering (ICSE)*, 2021, pp. 1572–1584.

[25] R. Verdecchia, E. Cruciani, B. Miranda, and A. Bertolino, "Know you neighbor: Fast static prediction of test flakiness," *IEEE Access*, pp. 76 119–76 134, 2021.

[26] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "Evaluating features for machine learning detection of order-and non-order-dependent flaky tests," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 93–104.

[27] Y. Qin, S. Wang, K. Liu, B. Lin, H. Wu, L. Li, X. Mao, and T. F. D. A. Bissyande, "Peeler: Learning to effectively predict flakiness without running tests," in *International Conference on Software Maintenance and Evolution (ICSME)*, 2022, pp. 1–12.

[28] S. Fatima, T. A. Ghaleb, and L. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE Transactions on Software Engineering*, pp. 1–17, 2022.

All online resources accessed on Friday, 9th of February 2023.